A

Project Synopsis

On

TaskFlow- Smart Workflow Management Platform

Submitted to

by

Vaibhav Annaso Chougule

Document Title: SMART WORKFLOW MANAGEMENT PLATFORM

Project Group: Group 4

Prepared By: Vaibhav Annaso Chougule
Branch: Information Technology

Approved By: Mr. Ketan Ganvir
Designation: Solutions Analyst

Submission Date: 16-03-2026

1. Problem Statement

Most people carry their tasks in their head assignments to submit, groceries to buy, meetings to attend.

- They forget things.
- They miss deadlines.
- They have no way to see what's done and what's pending.

A simple task manager fixes this.

The user types a task, sets a due date and priority, and can see everything clearly in one place. When they finish a task, they mark it done. That's it.

"Build an app where a user can add tasks, organize them by category, and track what's done."

This is a real app that people actually use every day. It covers every important full-stack concept: a database, a REST API, a dynamic frontend, and user interactions.

2. Existing System & Limitations

In daily life, most people manage their tasks using simple methods like writing in notebooks, using sticky notes, or setting reminders on their phones. While these methods help to remember some tasks, they are not very efficient when the number of tasks increases. Managing multiple tasks manually becomes difficult and often leads to confusion or missed deadlines.

Task Management

In the current system, people usually write the tasks in a notebook or notes app. There is no proper structure to manage tasks, set priorities, or track deadlines. Because of this, important tasks may be forgotten or completed late.

Lack of Organization

Traditional methods do not provide an easy way to organize tasks based on categories such as work, personal, or study. When many tasks are written together in one place, it becomes difficult to identify which task is more important or urgent.

Difficulty in Tracking Progress

In most existing systems, users cannot easily see which tasks are completed and which ones are still pending. They need to manually check and update their progress, which is time-consuming and sometimes inaccurate.

No Centralized Platform

Tasks are often written in different places such as notebooks, mobile reminders, or notes applications. Because of this, users do not have a single place where they can see and manage all their tasks together.

Limited Control and Flexibility

With manual methods, editing or updating tasks is not very convenient. If something changes, users may need to rewrite the task or create a new note, which makes task management inefficient.

3. Objective

The main objective of this project is to design and develop **TaskFlow – Task Manager Application** that helps users manage their daily tasks in a simple, organized, and efficient manner.

The system aims to provide a centralized platform where users can create, track, update, and manage their tasks easily.

The key objectives of this project are:

• **Allow users to register and log in securely** so that each user can manage their own tasks safely within the system.

• **Implement role-based access control (RBAC)** to manage permissions and ensure that only authorized users can perform certain operations.

• **Allow users to add and manage tasks easily**, including task title, description, due date, priority level, and category.

• **Provide a clear view of all tasks** so users can track their progress and identify which tasks are completed, pending, or overdue.

• **Enable users to update, mark complete, or delete tasks** whenever required, ensuring flexibility in managing daily activities.

• **Support filtering, sorting, and searching of tasks** so that users can quickly find tasks based on category, status, or keywords.

• **Implement pagination to handle large numbers of tasks efficiently**, improving application performance.

• **Provide a structured backend API system using REST architecture** to ensure smooth communication between frontend and backend components.

• **Store task and user data in a centralized database**, ensuring data consistency, security, and easy retrieval.

4. Scope of the Project

The scope of this project is to develop a **web-based TaskFlow application** that helps users manage their daily tasks in an organized and efficient manner. The system aims to provide a simple and secure platform where users can create, track, update, and manage their tasks easily.

The main scope of the project includes:

• The system will allow users to **register and log in securely**, ensuring that only authorized users can access the application.

• The system will support **role-based access control (RBAC)** to manage permissions and ensure secure access to system features.

• The system will allow users to **create and manage tasks** by entering details such as task title, description, due date, priority, and category.

• The application will provide a **web-based interface** where users can view all their tasks, track progress, and manage completed or pending tasks.

• The backend of the system will be developed using **Spring Boot**, which will provide REST APIs for handling all task-related operations such as adding, updating, deleting, and retrieving tasks.

• The system will use a **MySQL database** to store all user and task-related information securely.

• The system will support features such as **task filtering, searching, sorting, and pagination**, which will help users manage a large number of tasks efficiently.

• The frontend and backend will communicate through **REST APIs**, allowing smooth data flow between the user interface and the database

5. Proposed System Overview

The proposed system, TaskFlow – Task Manager Application, is designed to provide users with a centralized platform to organize and manage their daily tasks effectively. The system aims to reduce the difficulties of managing tasks manually and help users keep track of their work in a structured and efficient way.

TaskFlow allows users to create tasks, assign priorities, set due dates, and categorize tasks based on their type such as work, personal, or study. The system also allows users to update task status, mark tasks as completed, or delete tasks when they are no longer required.

Additionally, the system provides **secure user authentication and role-based access control**, ensuring that only authorized users can access and manage their tasks.

Key Objectives and Functionality

• **Task Management:** The system allows users to add, update, and delete tasks easily. Each task contains details such as title, description, category, priority, and due date.

• **Task Tracking:** Users can view all tasks in one place and easily track which tasks are completed, pending, or overdue.

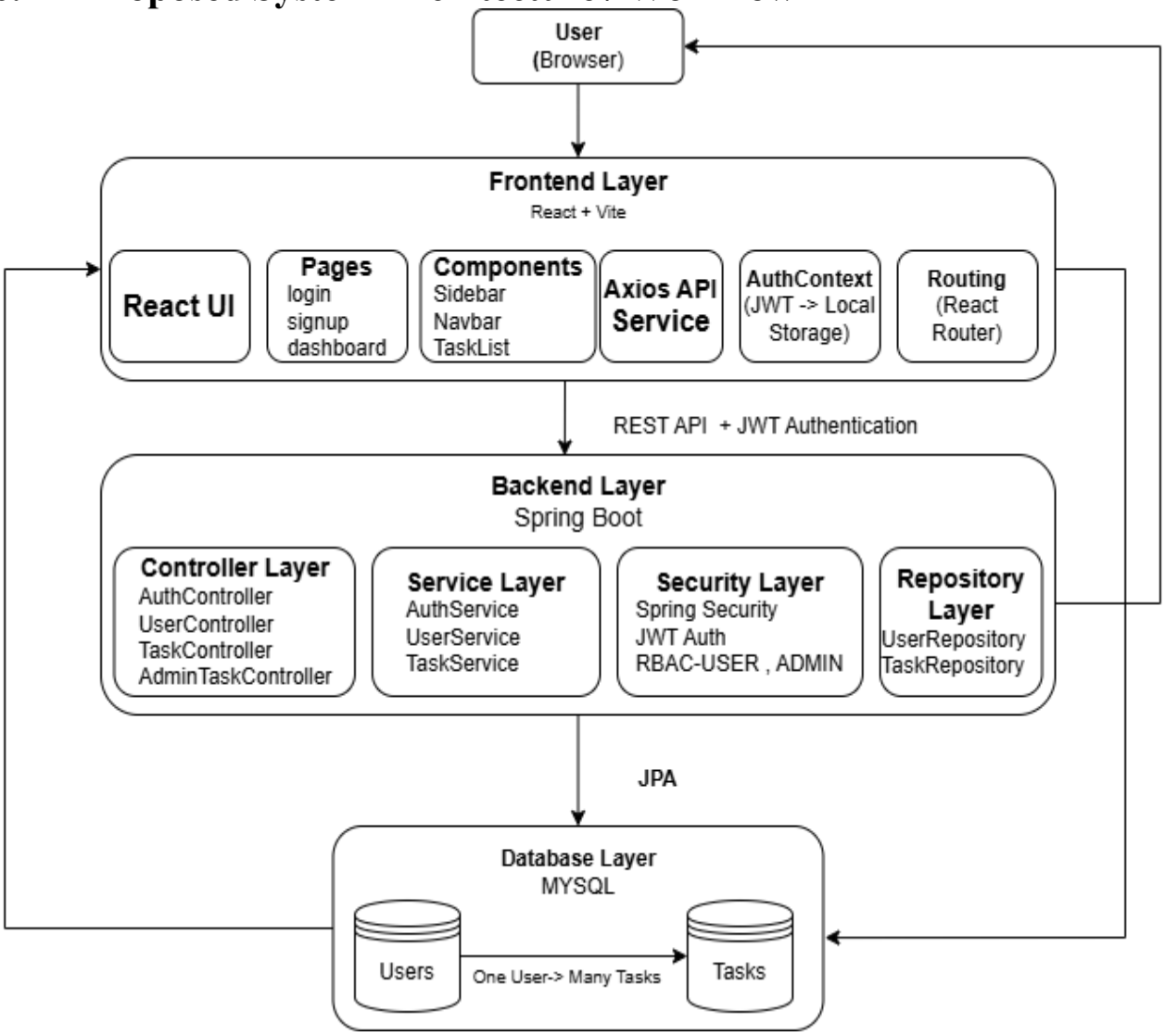
• **Search and Filtering:** The system provides filtering and searching features that help users quickly find tasks based on category, status, or keywords.

• **Efficient Data Handling:** Pagination and sorting features are implemented to handle large numbers of tasks efficiently and improve system performance.

• **Secure User Access:** The system includes authentication and authorization using JWT and role-based access control.

• **Centralized Data Storage:** All user and task data is stored in a database, ensuring that information is secure and accessible when required.

6. Proposed System Architecture / Workflow



The **TaskFlow system** follows a **three-tier architecture** consisting of the **Client Tier (Frontend)**, **Application Tier (Backend)**, and **Data Tier (Database)**. This architecture helps in separating the user interface, application logic, and data storage, making the system more scalable, secure, and easier to maintain.

In this architecture, users interact with the system through the frontend interface. The frontend communicates with backend services using REST APIs, and the backend processes requests and stores data in the database.

6.1 Architecture Components

1. Client Tier – Frontend (React)

The Client Tier represents the **user interface of the application**. It is developed using **React.js**, which provides a responsive and interactive web interface.

This layer is responsible for:

- Capturing user input
- Displaying task information to the user
- Communicating with backend APIs
- Managing UI components and navigation

Key Components:

- **UI Components:** Buttons, forms, tables, modals, and reusable UI elements.
- **Pages & Views:** Login, Dashboard, task list, add task form, and task details.
- **Frontend Services:** API services used to send and receive data from backend endpoints.

All user interactions such as creating tasks, updating tasks, or viewing task lists are initiated from this layer.

2. Application Tier – Backend (Spring Boot)

The Application Tier handles the **core functionality and business logic of the system**. It is implemented using **Spring Boot**, which exposes REST APIs for performing task-related operations.

The backend acts as a bridge between the frontend and the database.

Major Backend Components:

- **Security Layer:** Handles authentication using JWT and authorization using role-based access control (RBAC).
- **Controller Layer:** Handles incoming API requests from the frontend**.**
- **Service Layer:** Contains the business logic for managing tasks and users.
- **Repository Layer:** Communicates with the database using JPA/Hibernate.
- **Exception Handling:** Manages application errors using global exception handlers.

Backend Functionalities

a) User registration and login
b) JWT-based authentication
c) Role-based access control (Admin/User)
d) Task creation, update, deletion, and completion
e) Task creation, update, deletion, and completion

The backend ensures that only authorized users can access specific resources and that all operations follow defined business rules.

3. Data Tier – Database (PostgreSQL)

The Data Tier is responsible for **storing and managing all system data**. TaskFlow uses **MySQL**, which is a reliable relational database system.

Database Structure

Main tables used in the system**:**

• Users Table

- **id**
- **name**
- **email**
- **password**
- **role**
- **created_at**
- **reset_token**
- **is_enabled**

• Tasks Table

- **id**
- **title**
- **description**
- **category**
- **priority**
- **due_date**
- **is_done**
- **created_at**
- **user_id**

6.2 Workflow Summary

The following workflow explains how data flows through the system across all tiers:

1. User Registration and Login

Users register through the frontend by providing required information. After successful registration, they can log in using their credentials.
The backend validates the user and generates a **JWT token** for authentication.

2. Task Creation

After logging in, users can create new tasks by entering task details such as title, description, category, priority, and due date.
The frontend sends this data to the backend API, which stores the task in the database.

3. Task Management

Users can view all tasks on the dashboard, update task details, mark tasks as completed, or delete tasks when necessary.

4. Search and Filtering

Users can search tasks by keyword and filter tasks based on category or status. The backend processes the request and returns the filtered results.

5. Pagination and Sorting

To efficiently manage large numbers of tasks, pagination and sorting features are used so that tasks can be displayed in a structured and organized manner.

6.3 Architectural Benefits

- **Separation of Concerns:** Clear separation between frontend, backend, and database layers**.**
- **Scalability:** Each tier can be scaled independently
- **Security:** JWT authentication and role-based access control ensure secure access to system features
- **Maintainability:** Modular architecture makes it easier to update and add new features.
- **Performance:** Optimized API communication and database queries improve overall system performance.

7. Modules Description

7.1 User Authentication Module

The User Authentication Module is responsible for managing all user access and security-related operations within **TaskFlow**. This module ensures that only authorized users can access their personal tasks and that administrative features are restricted to authorized person role.

Purpose

The primary purpose of this module is to handle user onboarding, secure login, and identity verification, providing a protected environment for users to manage their tasks and for Administrators to oversee the platform.

Key Functions

- **Secure Registration:** Allows new users to create accounts with input validation and mandatory **OTP (One-Time Password) email verification** to ensure email authenticity.
- **Credential Validation:** Enables secure login using registered email and password with real-time error handling.
- **JWT Session Management:** Generates and manages **JSON Web Tokens (JWT)** to maintain stateless, secure user sessions across the React frontend.
- **Role-Based Access Control (RBAC):** Restricts system features based on user roles (e.g., **User** for personal task management and **Admin** for system-wide dashboards).
- **Account Security:** Supports secure password recovery via **Token-based Reset Links** and profile management.

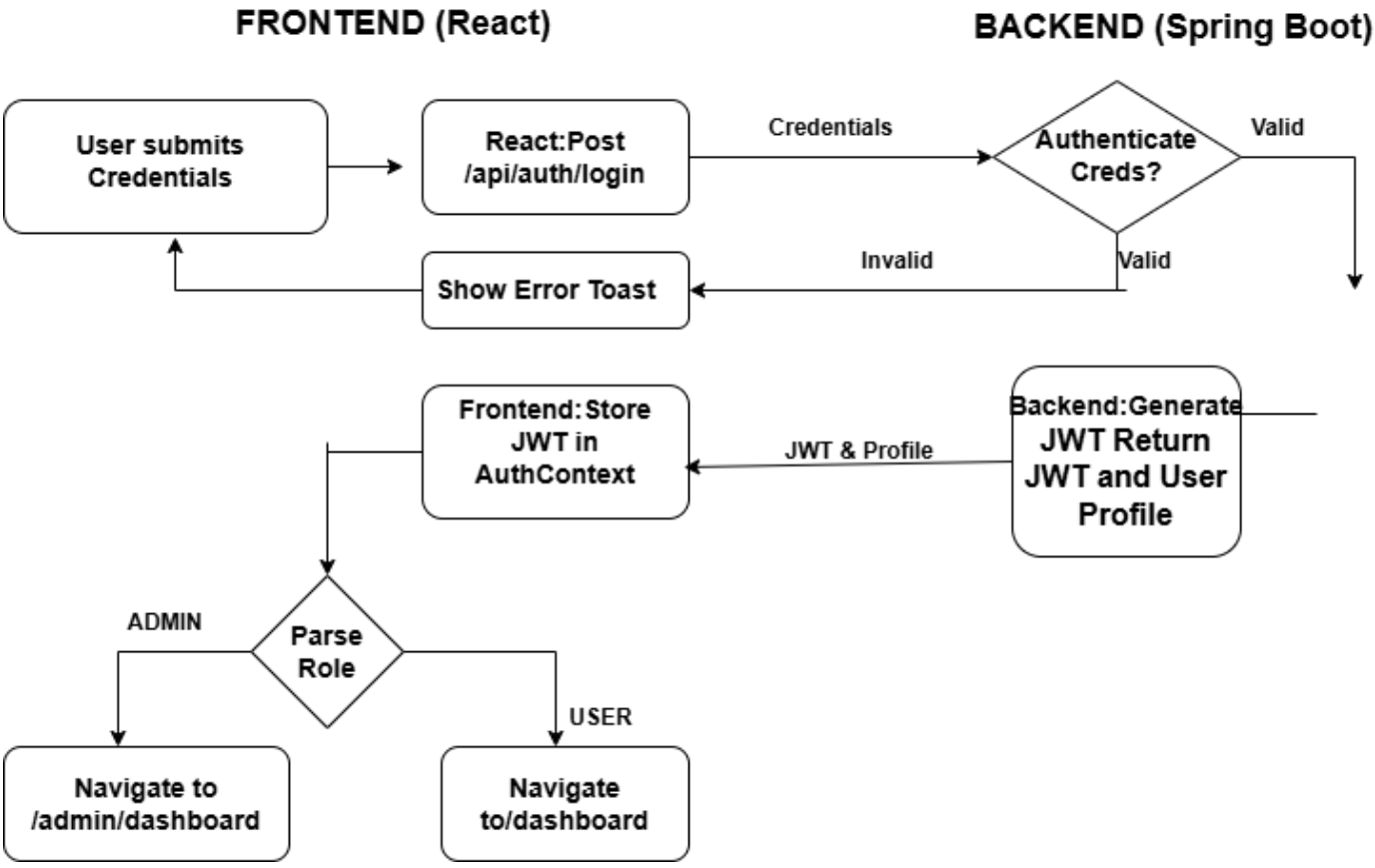
Process Flow

1. **Onboarding:** New users submit their details; the system sends a 6-digit OTP to their email.
2. **Verification:** Upon entering the correct OTP, the account is activated, and the user is redirected to the login page.
3. **Authentication:** Existing users log in; the Spring Boot backend validates credentials against the **MySQL** database.
4. Upon success, a JWT is generated and stored on the client side to authorize subsequent API requests.
5. **Authorization:** The system checks the user's role (Admin/User) and redirects them to the appropriate dashboard.

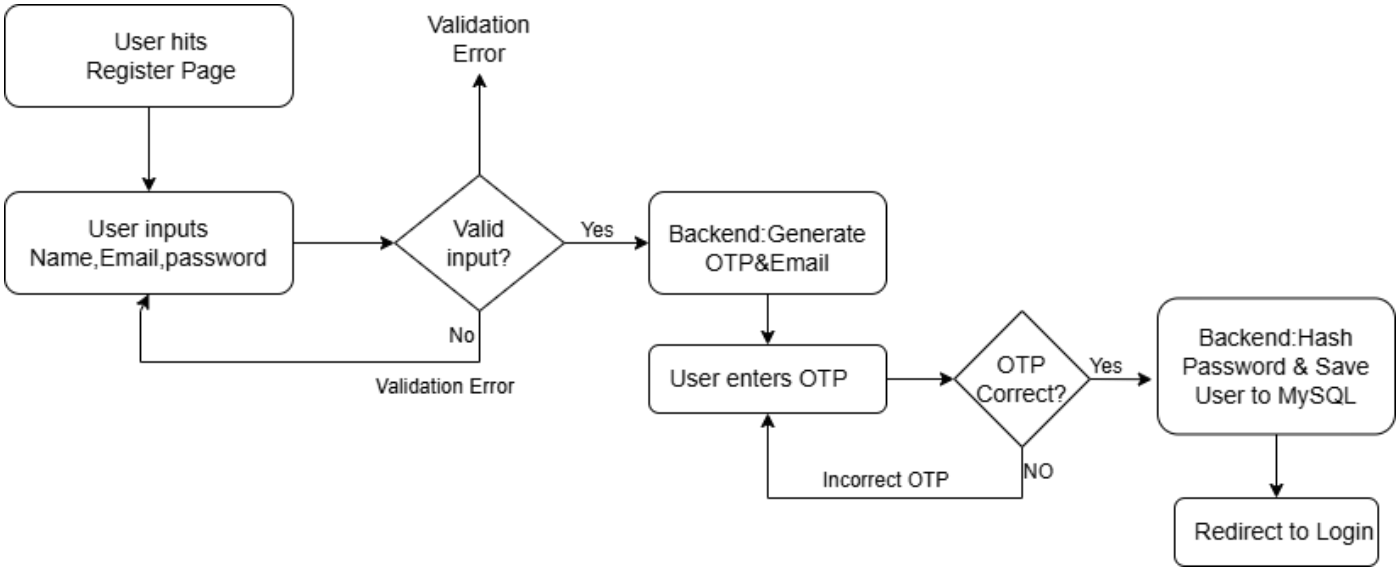
Module Outcome

This module ensures secure access, protects user privacy, and forms the foundation for all other task management features by enforcing strict authentication and authorization rules.

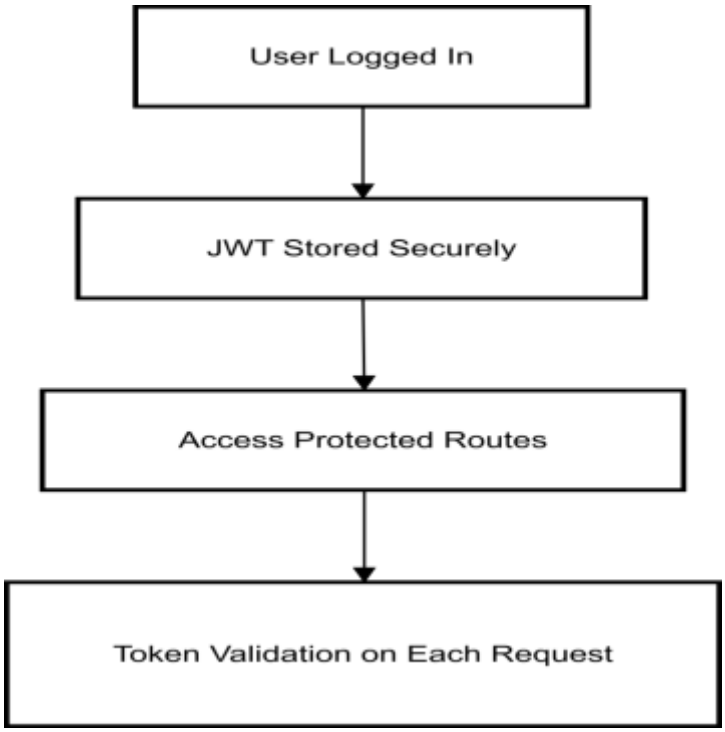
Module 1: User Login & Access Decision Flow



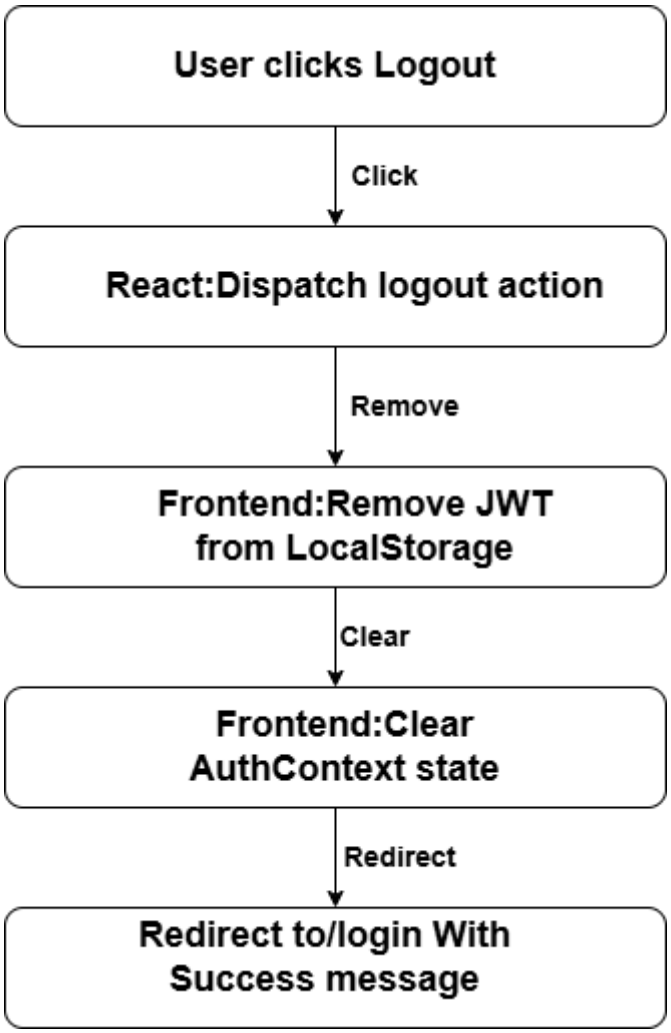
Module 2: User Registration Flow



Module 3: Session & Token Management Flow

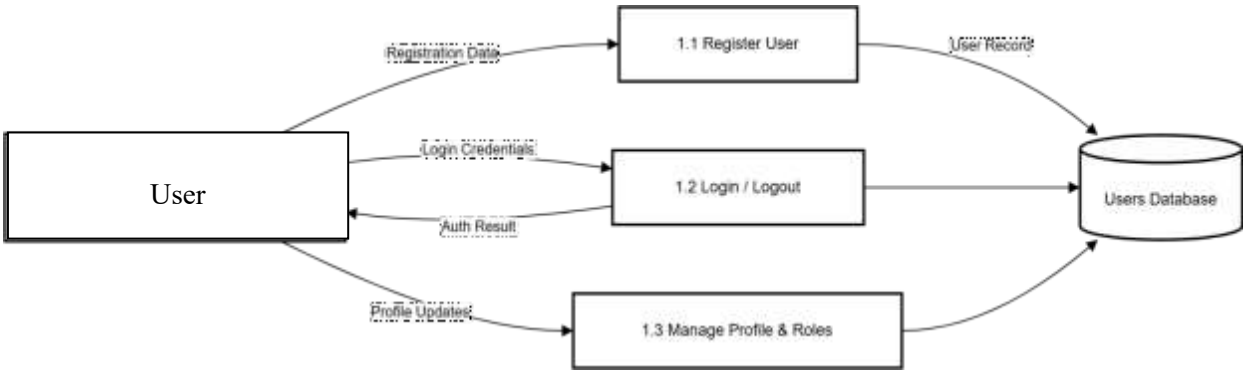


Module 4: Logout Flow



DFD Level 0:

DFD Level 1:


User
Registration Data
1.1 Register User
User Record
Login Credentials
1.2 Login / Logout
Auth Result
Users Database
Profile Updates
1.3 Manage Profile & Roles

7.2 Task Management Module

The Task Management Module is the core operational component of the **TaskFlow** system. It allows users to organize their workload, set priorities, and track the progress of their assignments from creation to completion. This module interacts directly with the **MySQL** database to ensure data persistence and real-time updates across the user dashboard.

Purpose

The primary purpose of this module is to provide users with a robust digital interface to manage their daily tasks, ensuring that deadlines are met and productivity is monitored through a structured workflow.

Key Functions

- **Task Creation (CRUD):** Enables users to generate new tasks by capturing essential data such as title, detailed description, category, and due dates.
- **Priority Ranking:** Allows users to assign priority levels (e.g., High, Medium, Low) to tasks to help focus on critical work first.
- **Status Tracking:** Supports real-time status updates (e.g., Pending, In-Progress, Completed), providing a visual representation of work stages.
- **Deadline Management:** Integrates due-date validation to help users track upcoming and overdue tasks.
- **Categorization:** Enables users to group tasks into specific domains (e.g., Work, Personal, Education) for better organization.
- **Data Integrity:** Performs server-side validation using Spring Boot to ensure that task data is consistent and linked correctly to the authenticated user's ID.

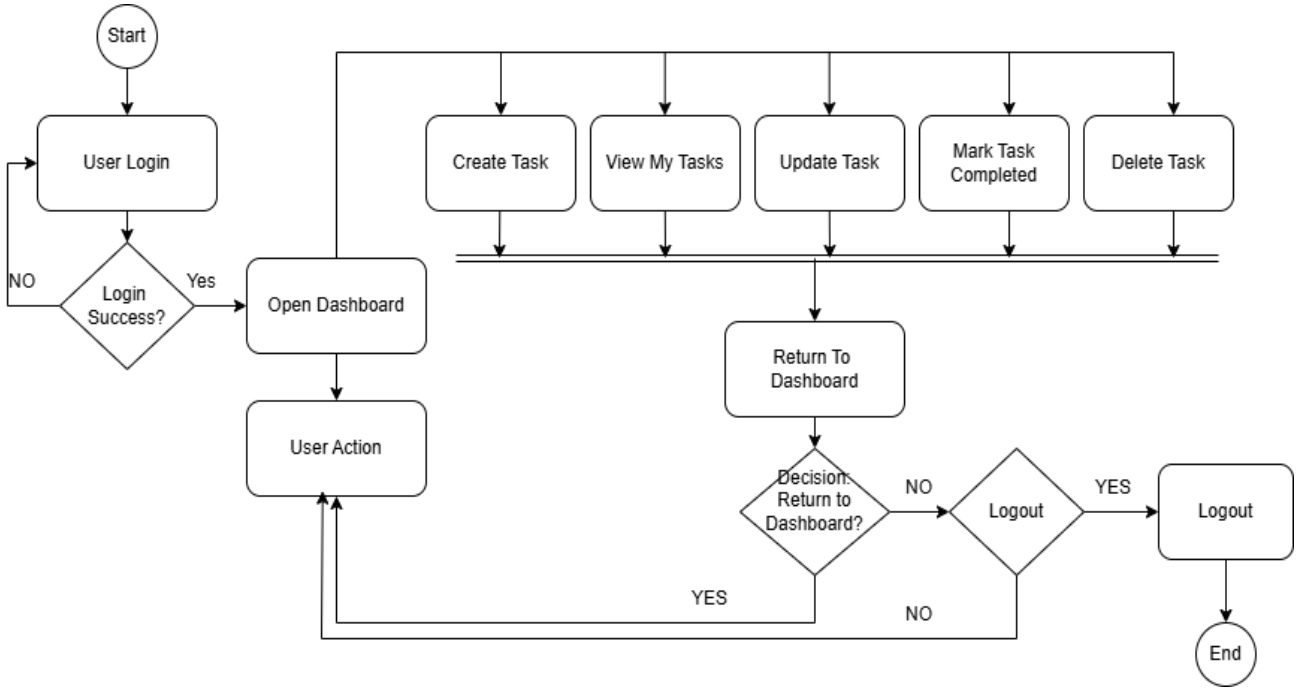
Process Flow

1. **Access:** After successful JWT authentication, the user accesses their personal Dashboard.
2. **Input:** The user opens the "Add Task" modal and enters the required task details and priority.
3. **Submission:** The React frontend sends a POST request to the backend API (/api/tasks) with the user's token in the header.
4. **Persistence:** The Spring Boot service validates the request and stores the task in the **Tasks Table**, mapped to the specific user.
5. **Interaction:** The user can later search, filter, or edit these tasks through the UI.
6. **Lifecycle Update:** When a task is finished, the user marks it as "Completed," triggering a PATCH or PUT request to update the status in the database.
7. **Deletion:** Users can remove tasks that are no longer required, which performs a secure delete operation in the backend.

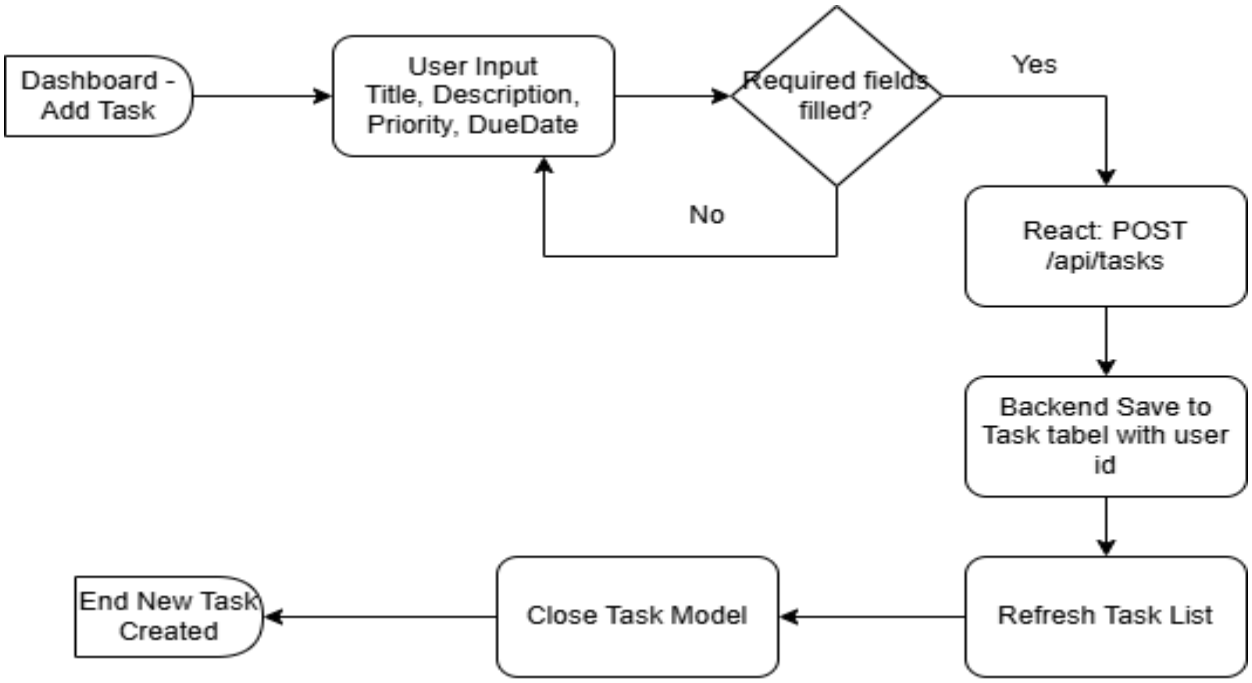
Module Outcome

This module ensures that all tasks are accurately recorded, easily searchable, and updated in real-time, providing the user with a comprehensive tool for effective time and workload management.

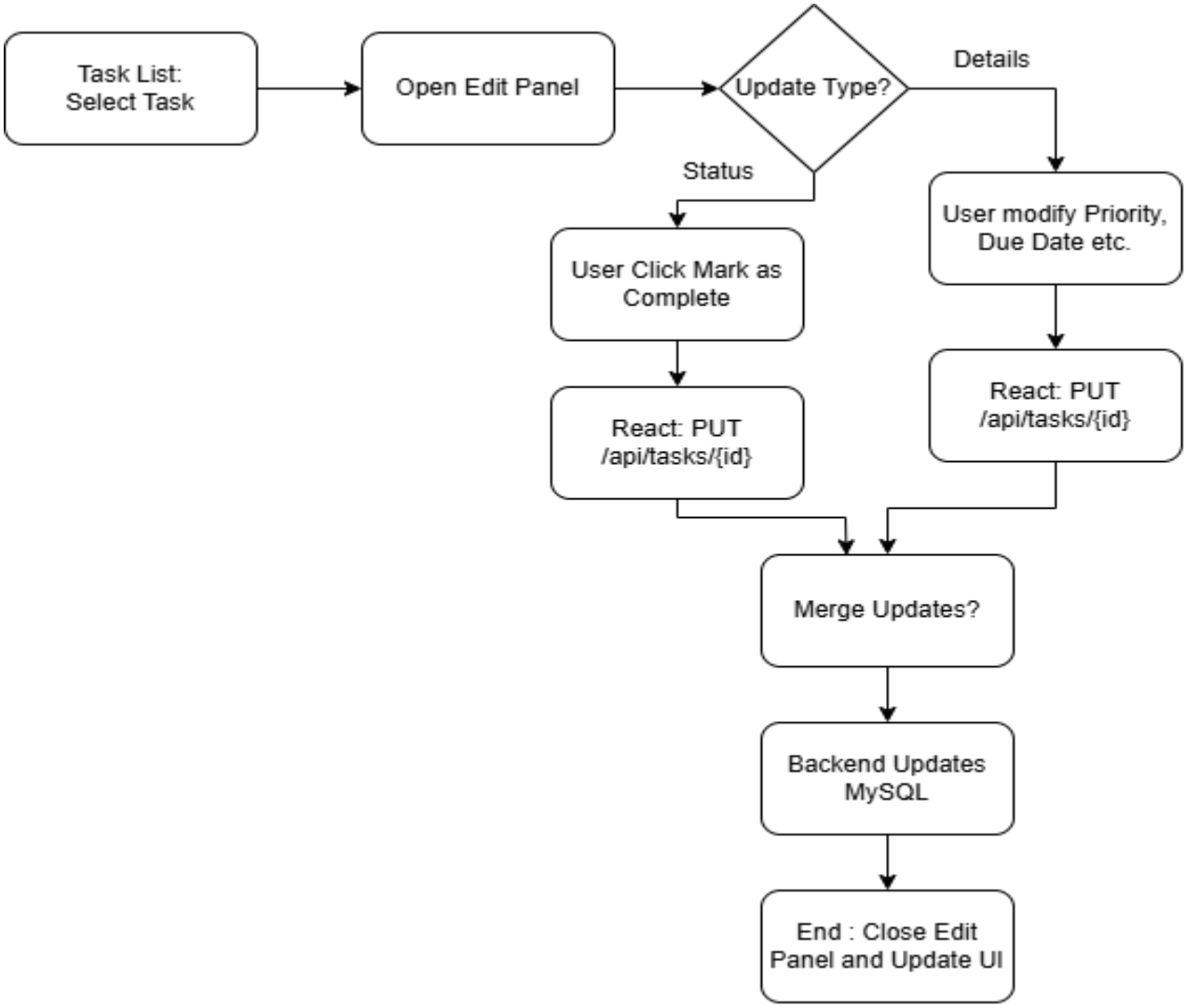
Module 1: Task Management Work Flow



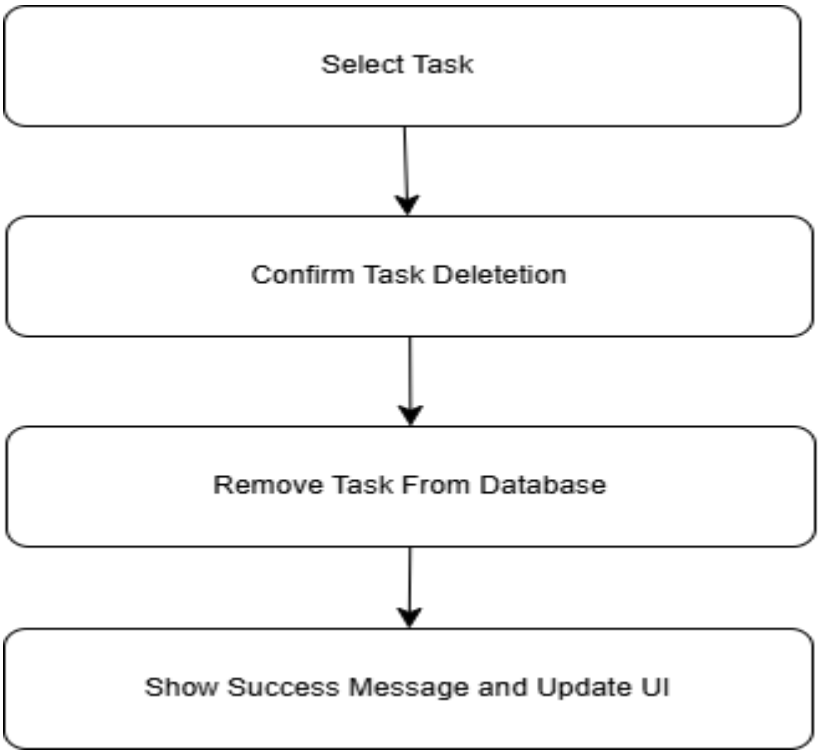
Module 2: User- Create New Task



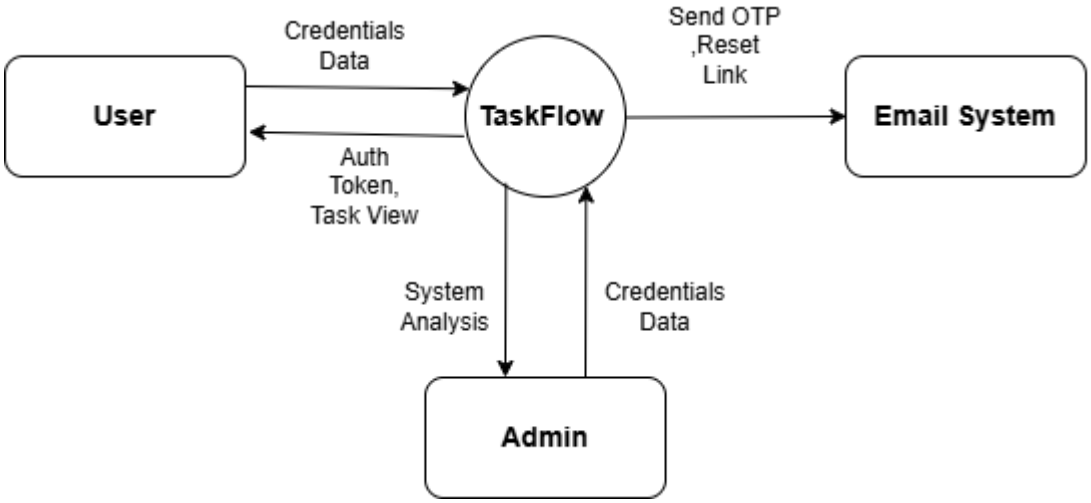
Module 3: User – Update Task Data or Mark Task as Completed Flow.



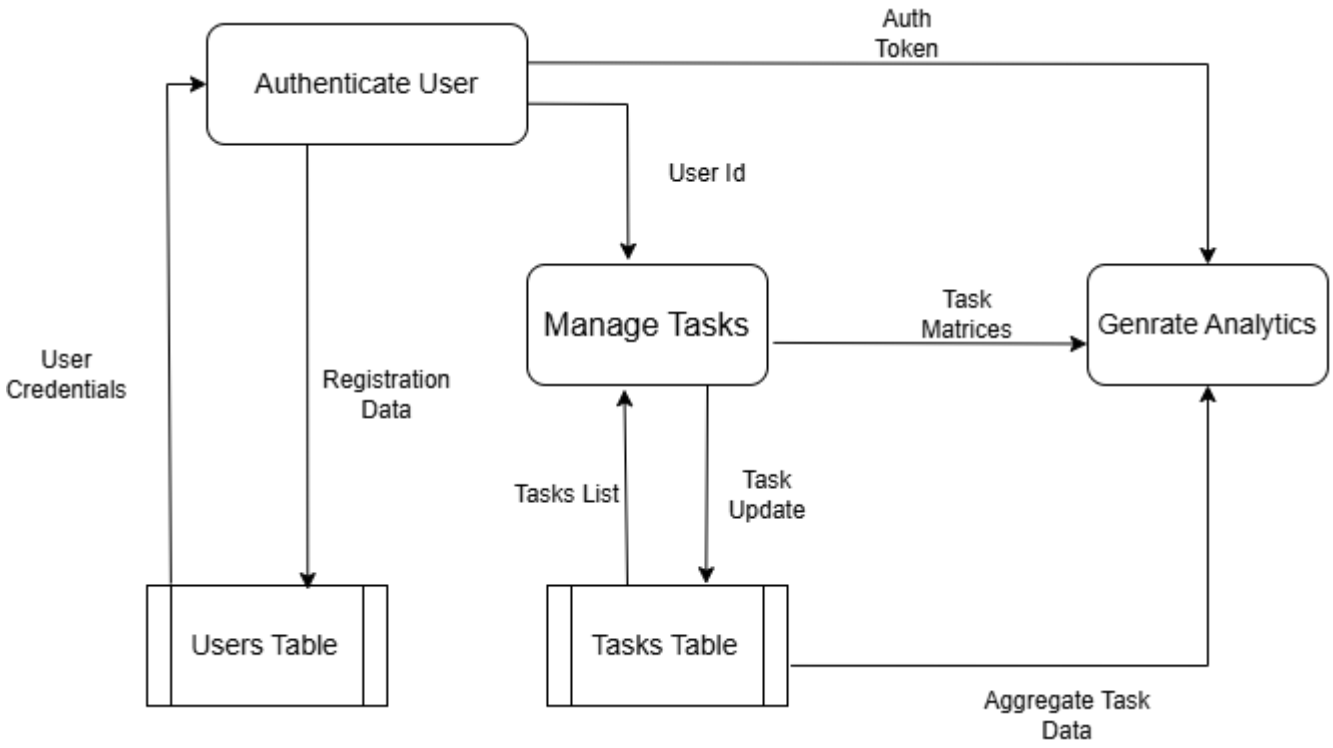
Module 4: User- Delete Task Flow



DFD Level 0:



DFD Level 1:


Authenticate User
Auth
Token
User Id
Manage Tasks
Task
Matrices
Genrate Analytics
User
Credentials
Registration
Data
Tasks List
Task
Update
Users Table
Tasks Table
Aggregate Task
Data

7.3 Admin Management Module

The Admin Management Module provides authorized administrators with the tools necessary to oversee system-wide activity, manage user accounts, and monitor global task distributions. This module serves as the central control hub for maintaining the health, security, and operational efficiency of the **TaskFlow** platform.

Purpose

The primary purpose of this module is to empower administrators to moderate the platform, manage user roles, and access high-level analytics that are not available to standard users. It ensures the system is being used correctly and provides a bird's-eye view of all platform data.

Key Functions

- **User Oversight:** Displays a list of all registered users, including their account status, roles, and registration dates.
- **Global Task Monitoring:** Provides a read-only or moderate-access view of all tasks created across the system to ensure platform guidelines are met.
- **System Analytics:** Aggregates data to show key metrics, such as the total number of active users, total tasks completed system-wide.
- **Search & Audit:** Allows administrators to search for specific users by email or ID to resolve technical issues or perform audits.
- **Role-Based Security:** Enforces strict server-side checks to ensure that only users with the ADMIN role can access the endpoints associated with this module.

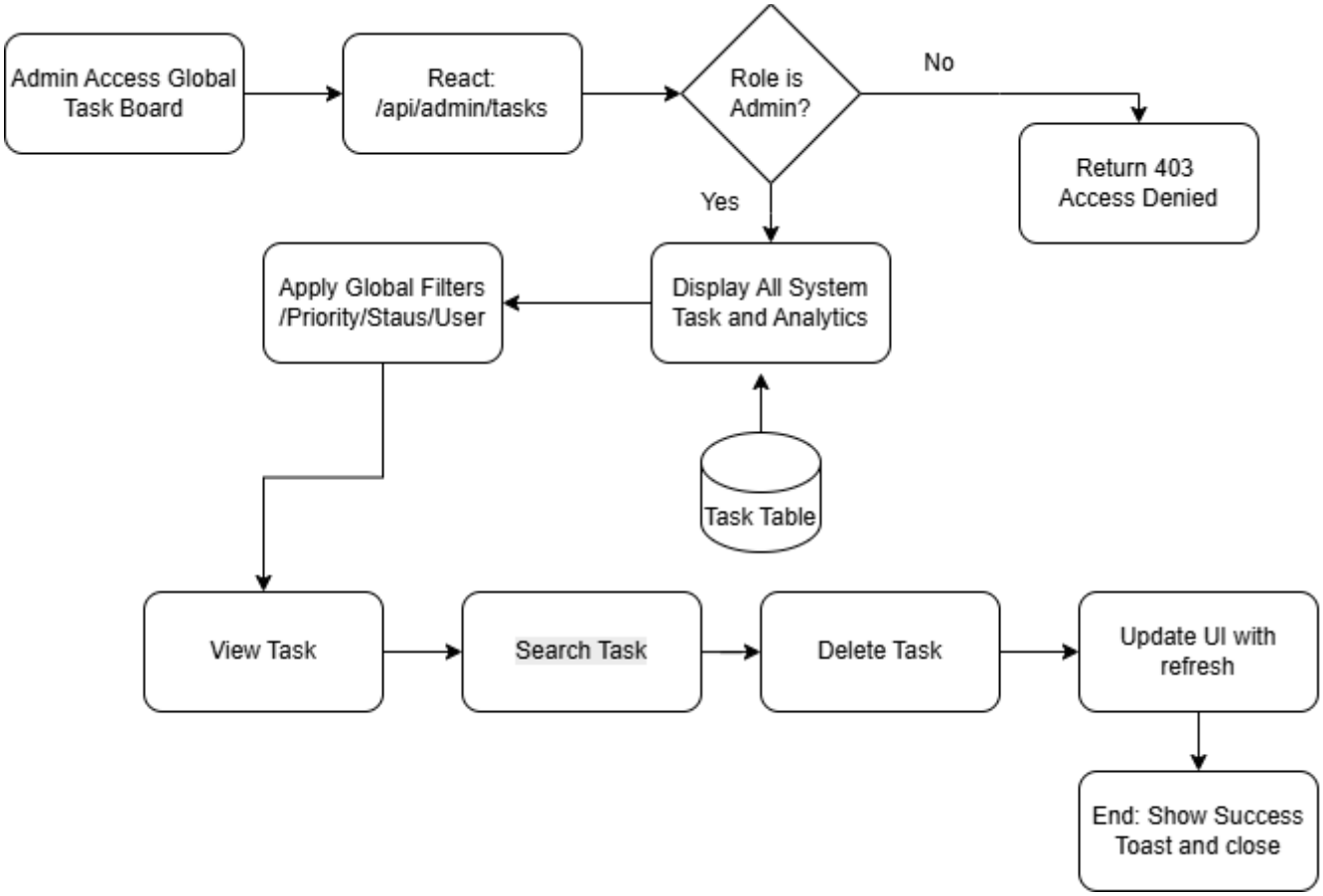
Process Flow

1. **Administrative Login:** The system detects the ADMIN role in the JWT payload upon login and redirects the user to the Admin Dashboard.
2. **User Management:** The admin navigates to the "User Management" section, where the frontend fetches all user records via a secured GET /api/admin/users request.
3. **System Observation:** The admin can view the "Task Overview" panel to see total task volume and status distribution (e.g., how many tasks are currently "Overdue" system-wide).
4. **Data Retrieval:** The Spring Boot backend processes these requests by querying the **Users** and **Tasks** tables and applying administrative filters.

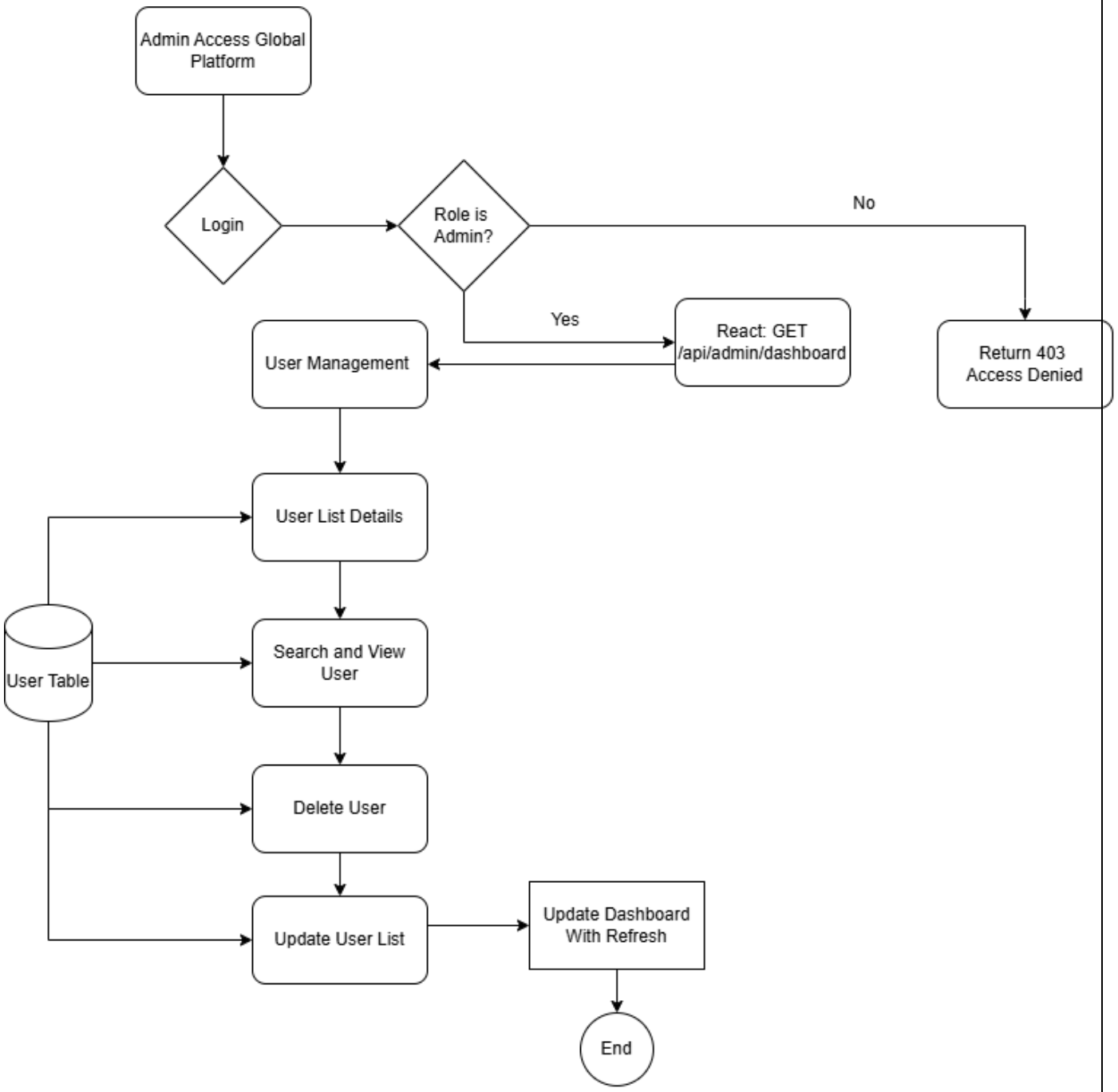
Module Outcome

This module ensures that the **TaskFlow** platform remains organized and secure. It provides the necessary transparency for administrators to manage the user base effectively and gain insights into how the system is being utilized at scale.

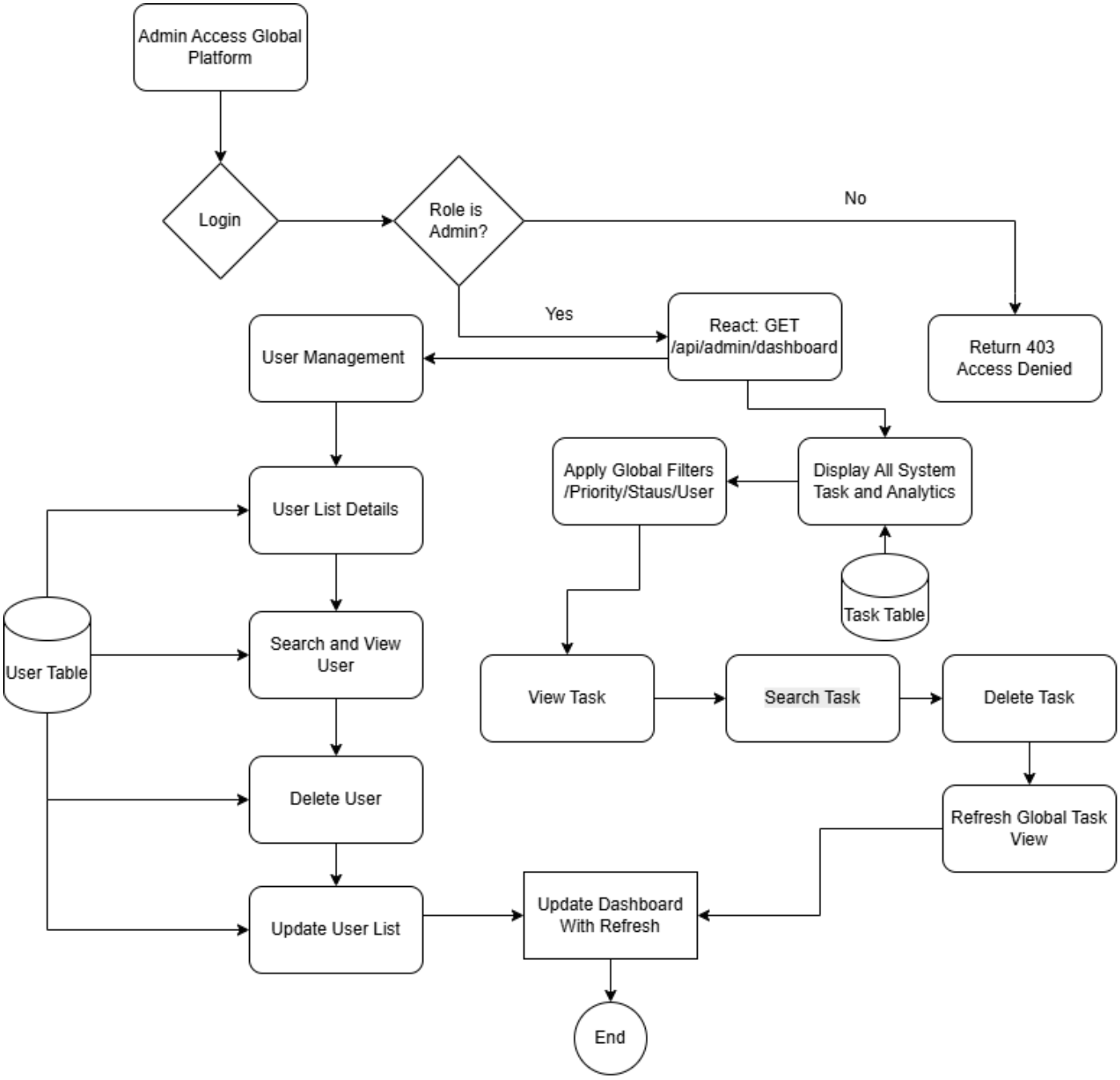
Module 1: Admin Task Management Flow



Module 2: Admin User Management Flow



Admin Activity Diagram:



8. Functional and Non-Functional Requirements

1. Functional Requirements

1.1 User Management

- **FR-01:** The system shall allow users to register with an email and password.
- **FR-02:** The system shall provide secure authentication via email/password validation and JWT tokens.
- **FR-03:** The system shall support Role-Based Access Control (RBAC) for **User** and **Admin** roles.
- **FR-04:** The system shall provide a "Forgot Password" flow utilizing OTP or secure reset links.

1.2 Task Operations (Core Logic)

- **FR-05:** The system shall allow users to create tasks with a Title (required), Description (optional), Category, Priority, and Due Date.
- **FR-06:** The system shall allow users to mark tasks as "Completed" or "Pending" via a simple toggle/checkbox.
- **FR-07:** The system shall allow users to delete tasks after a confirmation prompt.
- **FR-08:** The system shall allow users to edit existing task details (Title, Priority, Date).

1.3 View & Organization

- **FR-09:** The system shall categorize tasks into predefined groups: **Work, Personal,** and **Study**.
- **FR-10:** The system shall provide dynamic filters for **Today, Pending, Completed,** and **Overdue** tasks.
- **FR-11:** The system shall provide a search bar to filter tasks by title in real-time.

1.4 Admin & Analytics

- **FR-12:** The system shall allow Admins to view a global dashboard of all tasks and users.
- **FR-13:** The system shall generate productivity metrics, such as the percentage of tasks completed today.

2. Non-Functional Requirements

2.1 Performance & Scalability

- **NFR-01:** The system shall respond to API requests within **2 seconds** under normal load.
- **NFR-02:** The system shall support horizontal scaling via its stateless JWT architecture.
- **NFR-03:** Database queries shall be optimized with proper indexing.

2.2 Security

- **NFR-04:** All passwords shall be hashed using **BCrypt** before storage.
- **NFR-05:** The system shall use **HTTPS** to encrypt data in transit.
- **NFR-06:** The system shall sanitize all inputs to prevent **SQL Injection** and **XSS** attacks.

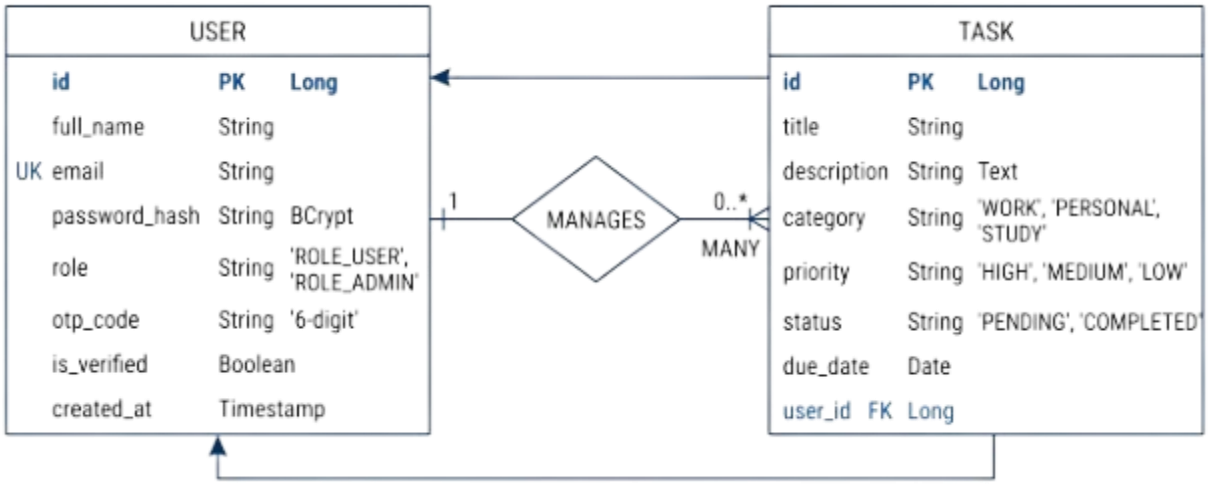
2.3 Usability & Design

- **NFR-07:** The UI shall be **fully responsive**, working seamlessly on Desktop, Tablet, and Mobile.
- **NFR-08:** The system shall use **dropdown selectors** and **date pickers** to minimize manual text entry errors.
- **NFR-09:** Monetary or percentage values shall omit .00 when the decimal is zero for a cleaner UI (e.g., **85%** instead of **85.00%**).
- **NFR-10:** The system shall provide **inline success/error messages** (Toasts) instead of intrusive modal dialogs.

2.4 Reliability & Maintainability

- **NFR-11:** The system shall maintain **99.5% uptime** for the duration of the pilot program.
- **NFR-12:** The codebase shall follow **Clean Code** principles (modular structure) to allow for independent component updates.

9. ER Diagram:


USER
id PK Long
full_name String
UK email String
password_hash String BCrypt
role String 'ROLE_USER', 'ROLE_ADMIN'
otp_code String '6-digit'
is_verified Boolean
created_at Timestamp
1
MANAGES
0..*
MANY
TASK
id PK Long
title String
description String Text
category String 'WORK', 'PERSONAL', 'STUDY'
priority String 'HIGH', 'MEDIUM', 'LOW'
status String 'PENDING', 'COMPLETED'
due_date Date
user_id FK Long

Entities and Their Descriptions

1. USERS

The **USERS** entity represents everyone registered on the TaskFlow platform.

- **Identity:** Users authenticate using email and password.
- **Roles:** Users are assigned roles (e.g., USER or ADMIN).
- **Ownership:** A User owns multiple tasks but a task can only belong to one User.
- **Session:** Users maintain their login state through tokens stored in the database.
- **Key Purpose:** Centralized identity management and role-based access control.

---

2. REFRESH_TOKENS

The **REFRESH_TOKENS** entity handles secure session management for the React frontend and Spring Boot backend.

- **Persistence:** Each token is linked to a specific user_id.
- **Security:** Allows the system to issue new JWT access tokens without requiring the user to re-login manually.
- **Relationship:** USERS authenticate_with REFRESH_TOKENS.

---

3. TASKS

The **TASKS** entity is the core of the application, representing the "To-Do" items created by users.

- **Metadata:** Stores the title, description, due_date, and is_done status (0 for Pending, 1 for Completed).
- **Logic:** Tracks when a task was created (created_at) to calculate if it is "Overdue" relative to the due_date.
- **Relationship:** USERS create TASKS. TASKS reference CATEGORIES and PRIORITY_LEVELS.

---

4. CATEGORIES

The **CATEGORIES** entity organizes tasks into logical groups as required by the F5 feature.

- **Types:** Predefined as Work, Personal, and Study.
- **Organization:** Helps users filter their dashboard to focus on specific life areas.
- **Relationship:** TASKS are_categorized_by CATEGORIES.

- **Key Purpose:** Improving user focus and task organization.

---

5. PRIORITY_LEVELS

The **PRIORITY_LEVELS** entity defines the urgency of tasks (F1 Requirement).

- **Levels:** Consists of High, Medium, and Low.
- **Visuals:** In the UI, these map to specific colors (Red, Amber, Green) for quick recognition.
- **Relationship:** TASKS have_assigned PRIORITY_LEVELS.

---

6. AUDIT_LOGS (Admin Module)

The **AUDIT_LOGS** entity represents the "Admin Flow" for oversight (Section 7.3).

- **Tracking:** Records actions taken by an Admin, such as "User Deactivation" or "Task Moderation."
- **Security:** Stores the timestamp and the ID of the Admin who performed the action.
- **Relationship:** ADMINS (Users) generate AUDIT_LOGS.
- **Key Purpose:** Accountability and system-wide security monitoring.

---

7. SYSTEM_STATS (Analytics Module)

The **SYSTEM_STATS** entity is a virtual/summary entity used for the Section 8.0 Overview Dashboard.

- **Aggregation:** Calculated data showing "Total Tasks," "Completion Percentage," and "Overdue Count."
- **Reporting:** Fed into the "Progress Ring" and "Stat Cards" on the UI.
- **Key Purpose:** Transforming raw task data into actionable productivity insights.

10. Hardware & Software Requirements

The **TaskFlow** system requires two distinct environments: the **Development Environment** for active coding and the **Production/Deployment Environment** for hosting the live application.

A. Software Requirements

- **Operating Systems:** Windows 10/11, macOS, or Linux.
- **Web Browsers:** Latest versions of **Google Chrome**, **Mozilla Firefox**, or **Microsoft Edge** to support the React-based client interface.
- **Backend Runtimes: * Java Development Kit (JDK) 17+**: Required for the Spring Boot backend.
- **Spring Boot 3.x**: Core Java framework for REST API development.
- **Maven**: For dependency management and build automation.
- **Frontend Environment**: **Node.js (v18+)** and **npm** for managing the React.js build process and Vite.
- **Database Systems: MySQL 8.0+** (Relational Database Management System).
- **Development Tools:**
    - **VS Code and Eclipse IDE** – for coding and project development.
    - **Postman** – For testing and validating REST API endpoints.

B. Hardware Requirements

- **Development Machine:**
    - Laptop or PC with at least an **Intel i5 processor**.
    - Minimum **8 GB RAM** for development and testing activities.
- **Client Devices:**
    - Desktop computers, tablets, and smartphones for accessing the system interface.

11. Expected Outcomes

The successful implementation of the TaskFlow system is expected to deliver significant improvements in personal and professional productivity management:

1. **Increased Efficiency:** Automation of task prioritization, category filtering, and status tracking will reduce cognitive load and organization time, potentially improving individual productivity by up to **50%**.
2. **Enhanced User Experience:** A modern, responsive React interface combined with real-time feedback (Toasts) ensures a seamless experience across all devices, from mobile to desktop.
3. **Improved Deadline Compliance:** The "Overdue" tracking logic and clear due-date visualizations will significantly reduce missed deadlines and forgotten assignments.

4. **Data-Driven Productivity:** The **Admin Dashboard** and User Analytics provide key insights into completion rates and workload distribution, allowing users to identify and resolve productivity bottlenecks.
5. **Secure Digital Record-Keeping:** By utilizing **JWT Authentication** and **MySQL**, all task histories and user data are stored securely, ensuring data integrity and easy retrieval for performance reviews or audits.

12. TaskFlow Project Implementation Plan

Project Overview

The **TaskFlow Management System** is a comprehensive, full-stack web application designed to streamline daily workflows and task organization. It supports multiple user roles (**User** and **Admin**) and provides features such as dynamic task CRUD, role-based analytics, and secure session management.

This project demonstrates professional-grade development skills using the following modern tech stack:

- **Frontend:** React.js (Vite) with Tailwind CSS for a modern UI.
- **Backend:** Java with Spring Boot for robust, scalable REST APIs.
- **Security:** Spring Security & JWT for stateless, secure authentication.
- **Database:** MySQL for high-performance relational data management

.

Project Timeline: 15 Days Total

Day	Phase	Focus Area	Detailed Activities
Day 1	Analysis & Planning	Requirement Analysis	Study existing task management tools, identify user needs, define functional & non-functional requirements, and finalize project scope.
Day 2	Backend Development	Project Setup & Entities	Initialize Spring Boot project, configure MySQL database, and implement User, Tasks, and Role entities with Repository layers.

Day 3	Backend Development	Task CRUD & Error Handling	Develop Task CRUD APIs (Create, Read, Update, Delete). Implement a Global Exception Handler to manage system errors gracefully.
Day 4	Backend Development	Performance & Sorting	Implement Pagination to handle large datasets. Add sorting logic to organize tasks by priority and update API documentation.
Day 5	Backend Development	Advanced Search	Develop specialized **Search REST APIs** using keywords to allow users to find specific tasks dynamically.
Day 6	Backend Development	Security & Auth	Setup **JWT-based authentication** and Spring Security. Integrate combined logic for filtering, sorting, and searching into secure endpoints.
Day 7	Frontend Development	Project Setup	Set up **React/Vite** project, configure protected routing, and implement the Authentication Context for session management.
Day 8	Frontend Development	Auth & RBAC	Build Login and Registration forms. Implement **Role-Based Access Control (RBAC)** to separate User and Admin dashboard views.
Day 9	Frontend Development	Admin Dashboard	Build the Admin Dashboard with analytics cards. Connect backend APIs to display system-wide user activity and task stats.
Day 10	Frontend Development	Data Visualization	Integrate **Bar graphs and Pie charts** on the Admin Dashboard for visual data analysis and user growth tracking.
Day 11	Frontend Development	User Profile & UX	Develop the **User Profile Page**. Enhance the main dashboard UI to show properly filtered tasks based on their current status.
Day 12	Frontend Development	Landing Page & UI	Build the **Landing page** and navigation. Implement task detail popups and ensure the application is fully responsive.

Day 13	Frontend Development	Personalization	Implement **Profile image upload** and avatar display. Add popup notifications for login and registration success feedback.
Day 14	Security & Features	Hardening & UX	Implement **OTP Email Verification** and Forgot Password flow with Spring Mail. Add **Account Deletion** feature and **Toast messages**.
Day 15	Testing & Documentation	Quality QA & Docs	**Project Testing:** Perform end-to-end validation of all flows. **Finalize:** Remove admin email verification, update README diagrams, and complete documentation.

13. Future Scope:

- **Collaboration & Team Tasks:** Enable users to share specific task categories or individual tasks with others for collaborative project management.
- **Real-time Push Notifications:** Integrate web-push notifications to alert users about upcoming deadlines or overdue tasks even when the browser is closed.
- **Calendar Integration:** Synchronize **TaskFlow** with external calendars like Google Calendar or Outlook to provide a unified view of schedules.
- **AI-Powered Prioritization:** Implement a machine learning module to suggest task priority levels based on the user's past completion habits and upcoming deadlines.
- **Advanced Productivity Reports:** Expand the Admin and User dashboards to include downloadable PDF reports and monthly productivity trends.

14. Conclusion

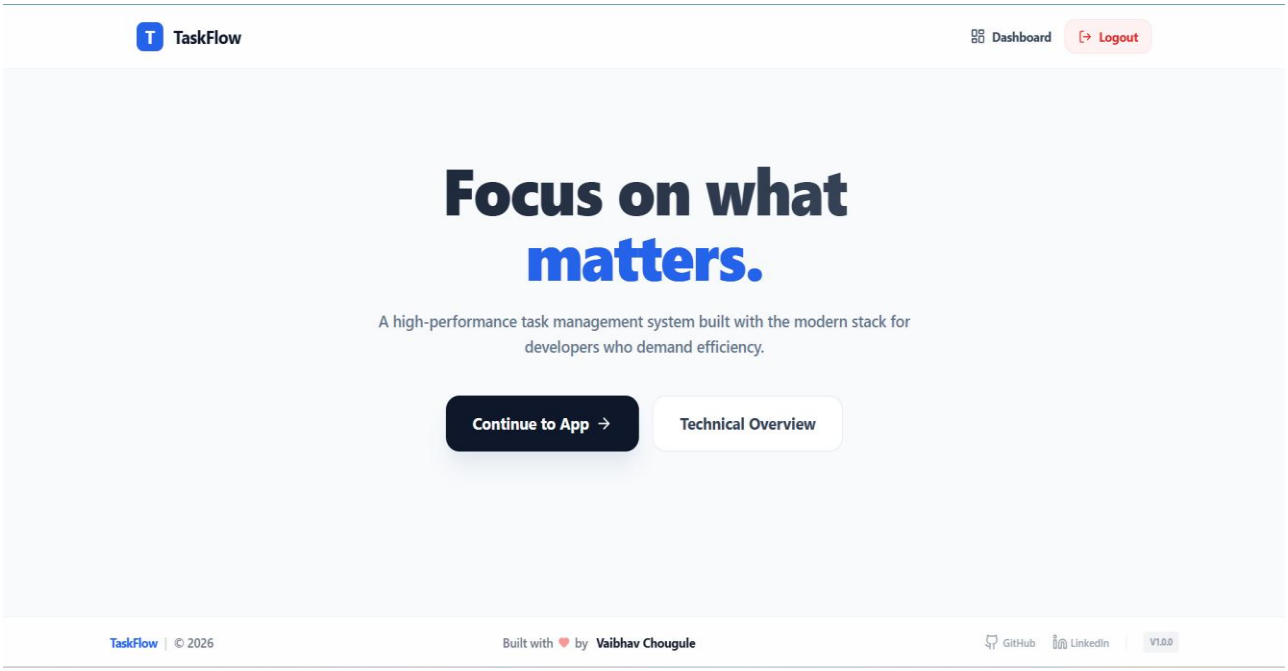
Modernizing personal and professional task management is essential in a fast-paced environment; however, the TaskFlow Management System provides a scalable and efficient solution using a robust full-stack tech stack of React.js, Spring Boot, and MySQL.

Traditional methods of tracking work—such as manual lists or basic spreadsheets—often result in missed deadlines, poor organization, and a lack of clear priority. These inefficiencies can lead to decreased productivity and increased stress.

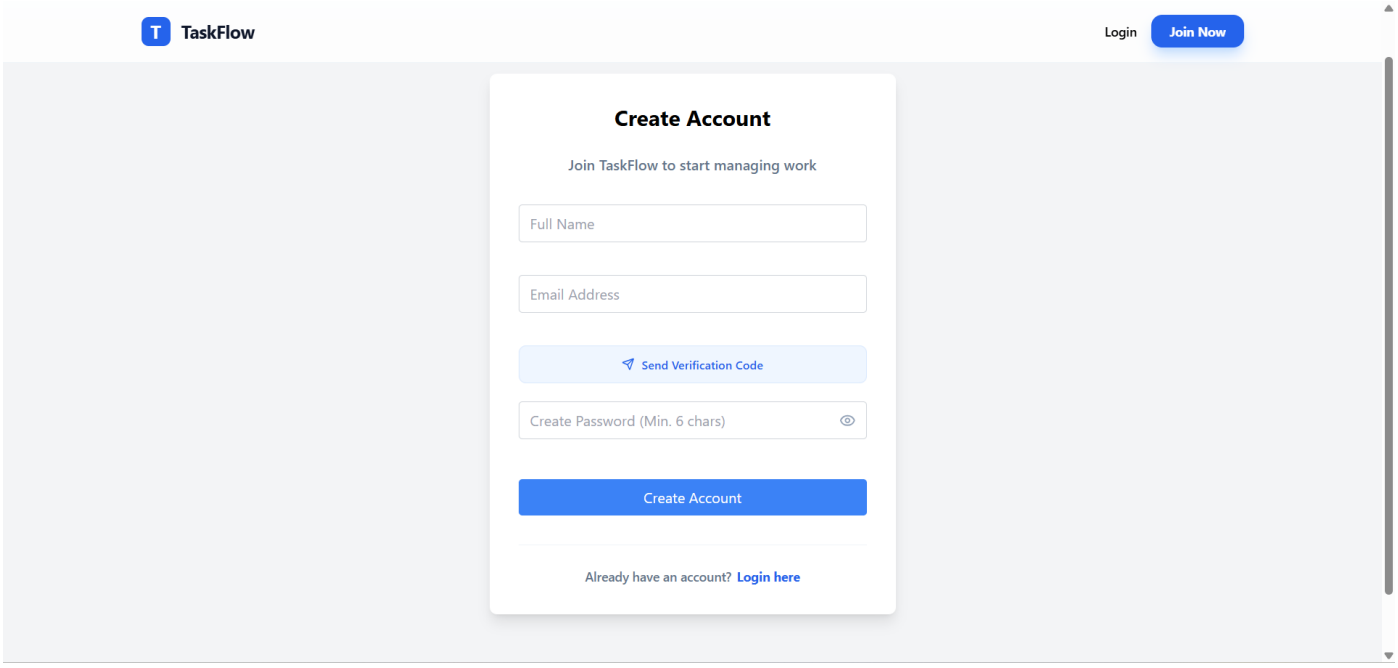
By implementing TaskFlow, users can automate their workflow, maintain organized digital records, and gain real-time insights into their progress through the analytics dashboard. The system enhances focus, ensures data accuracy, and improves overall time management, leading to better resource utilization and a more disciplined approach to achieving daily goals.

15. UI Screens:

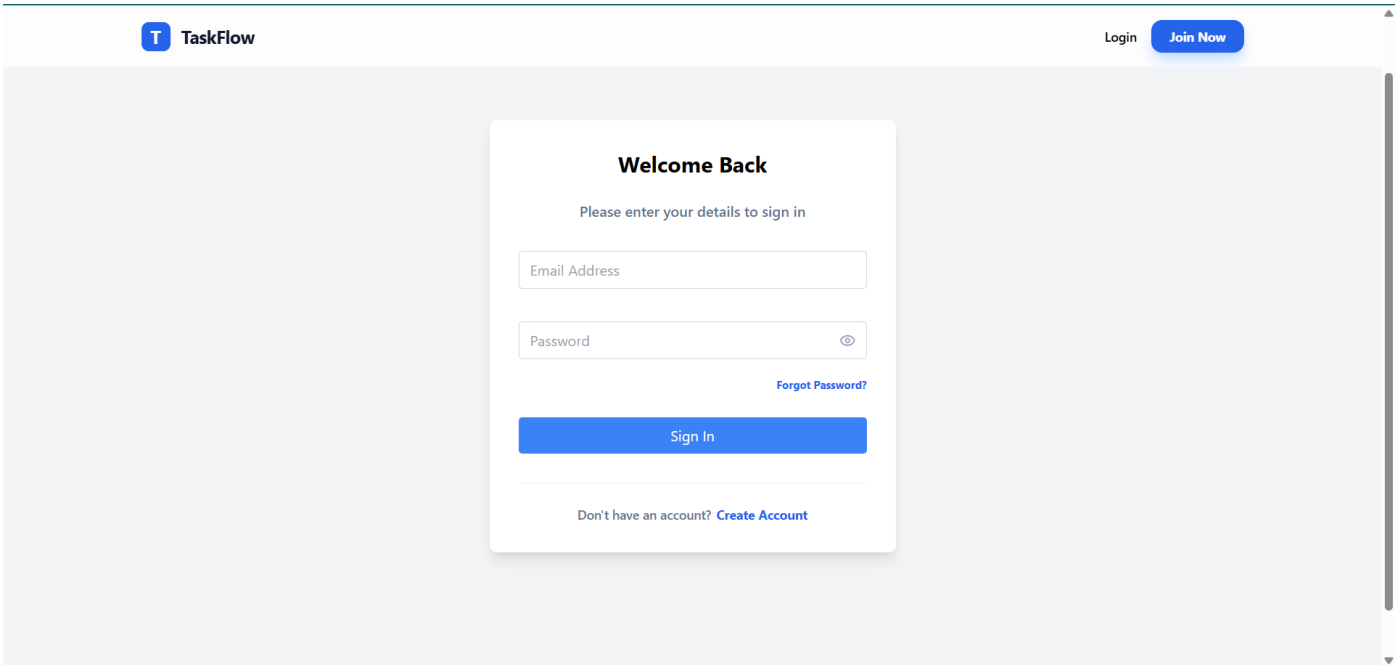
1. Home Page:



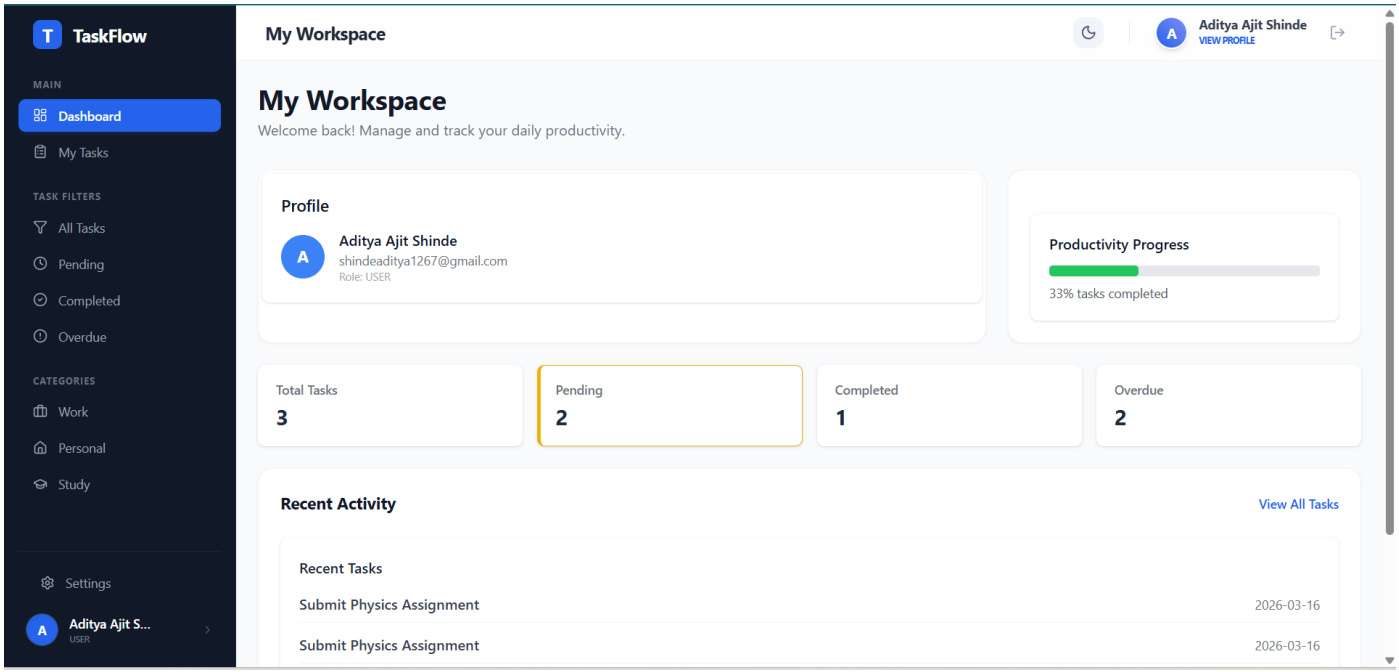
2. Sign Up Page:



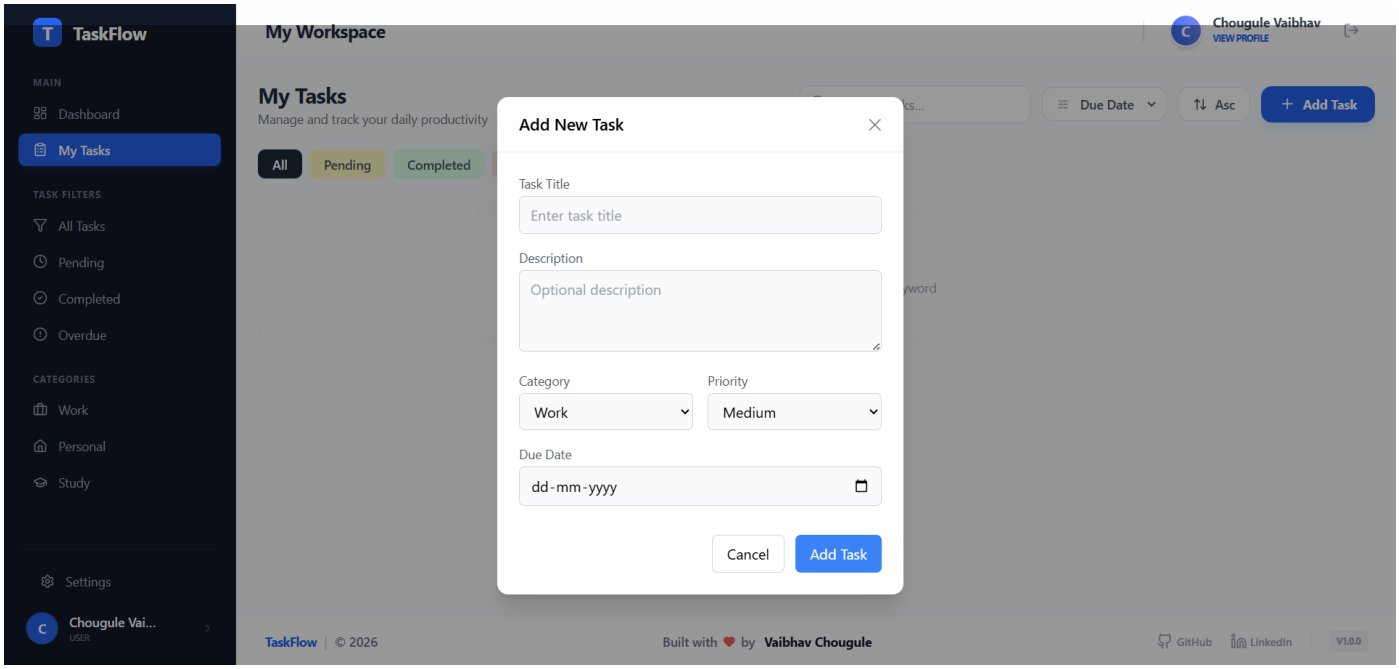
3. Login Page:


TaskFlow
Login
Join Now
Welcome Back
Please enter your details to sign in
Email Address
Password
Forgot Password?
Sign In
Don't have an account? Create Account

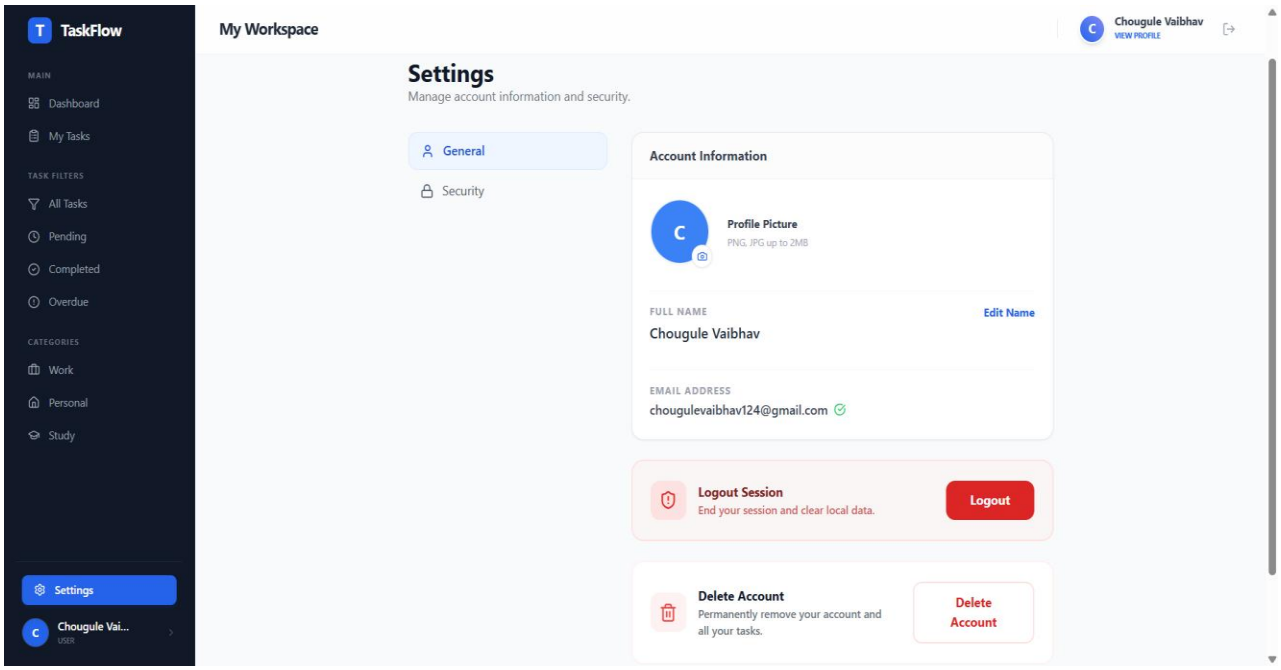
4. User Dashboard Page:



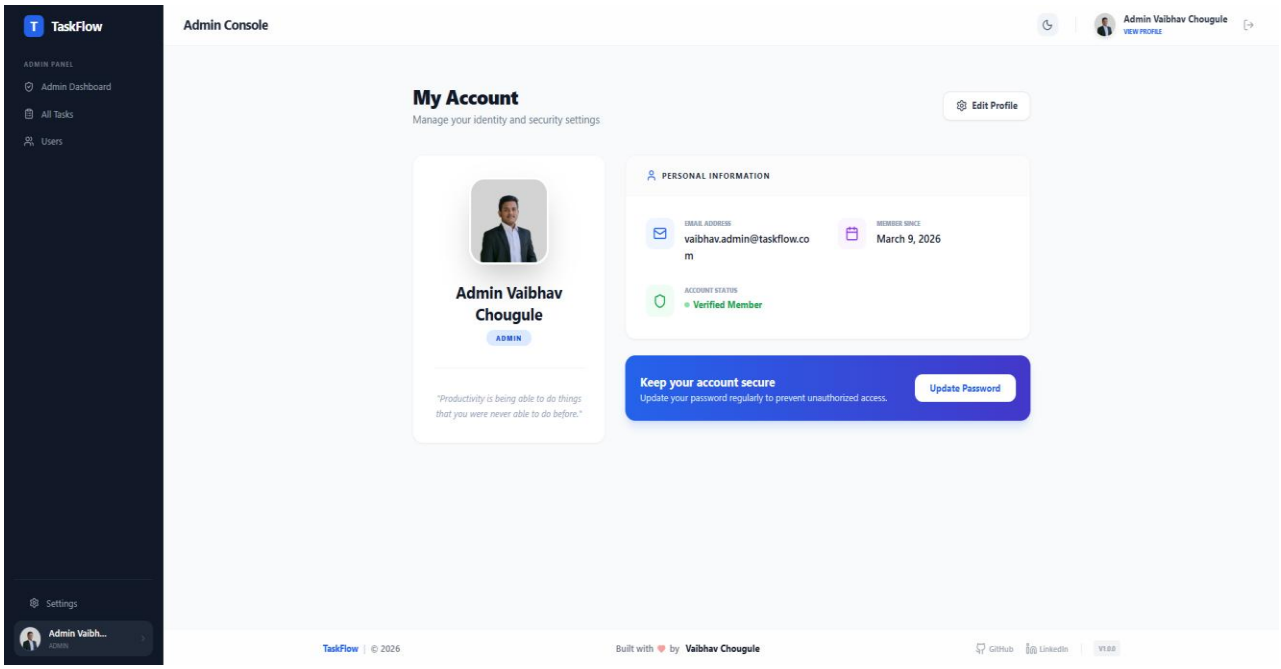
5. Customer Add Task Page:



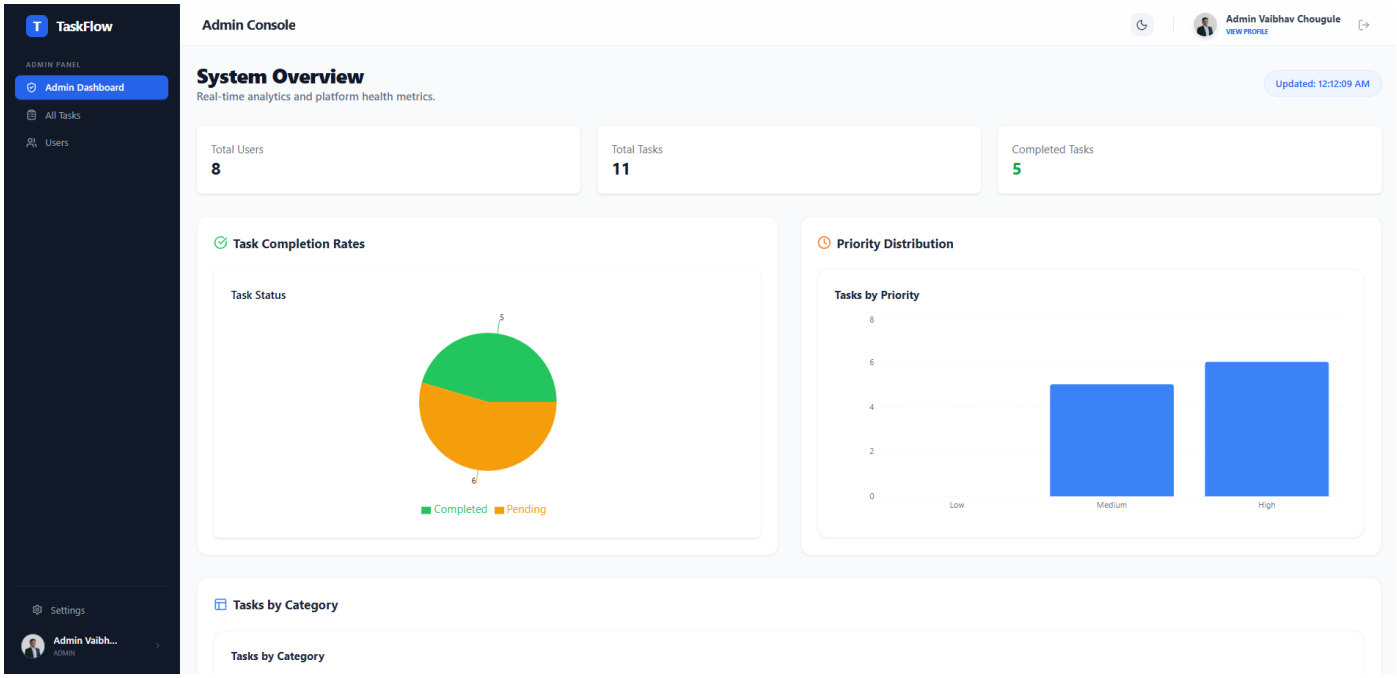
6. User – Admin Settings Page:



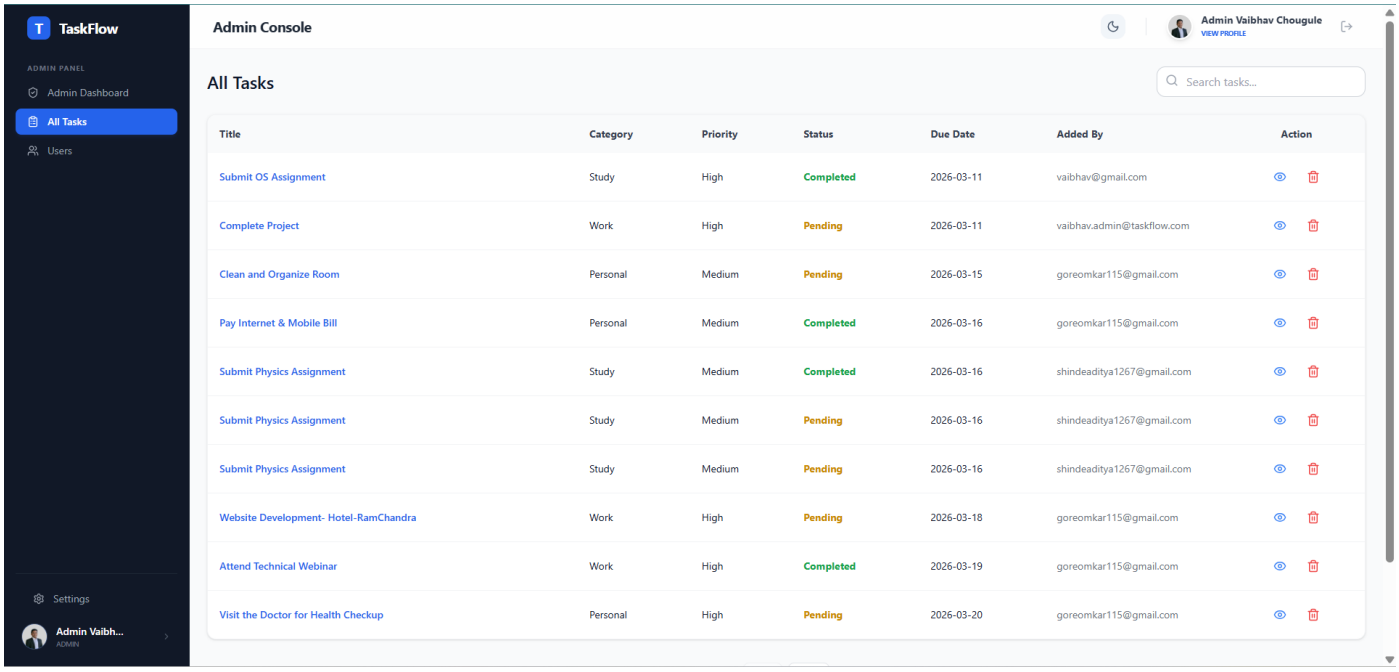
7. User – Admin Profile Page:



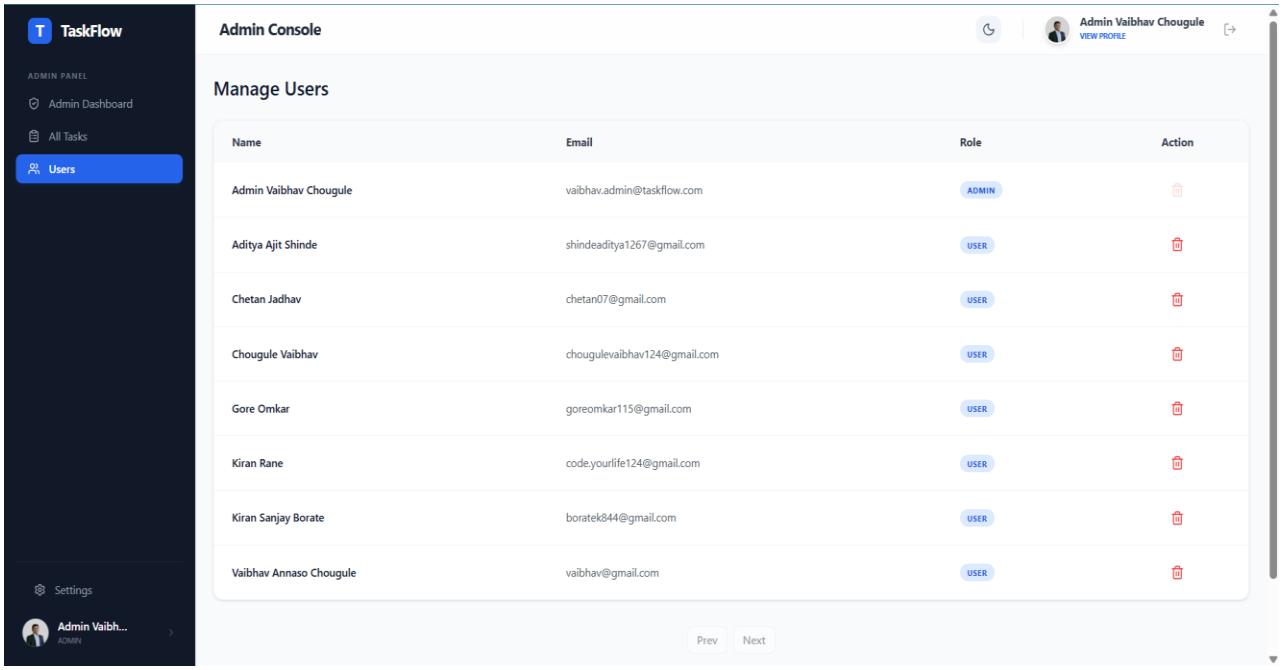
8. Admin Dashboard Page:



9. Admin Task Management Page:



10. Admin User Management Page:



11. Admin Analytics Page: